CS & IT ENGINEERING

Compiler Design

Intermediate code and optimization

Lecture No.- 02

By- Venkat sir

Recap of Previous Lecture

Topic — Intermediate Code Generation

Topic — Syntax tree

Topic — DAG

Topic

Topics to be Covered

Topic — Three address Code

Topic — Implementing TAC

Optimization

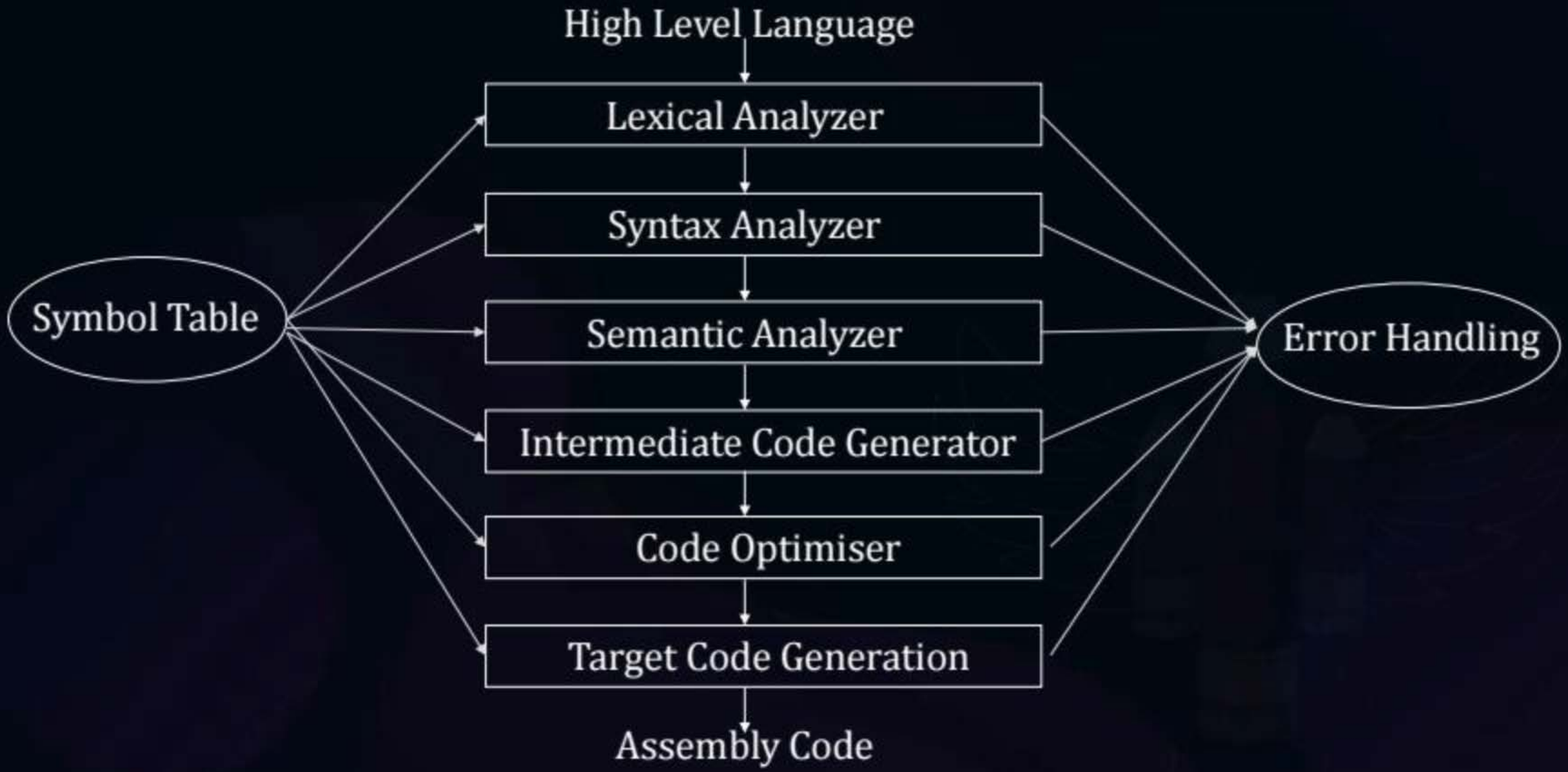
Basic Block

Control flow graph

Data flow Analysis

Runtime Environment

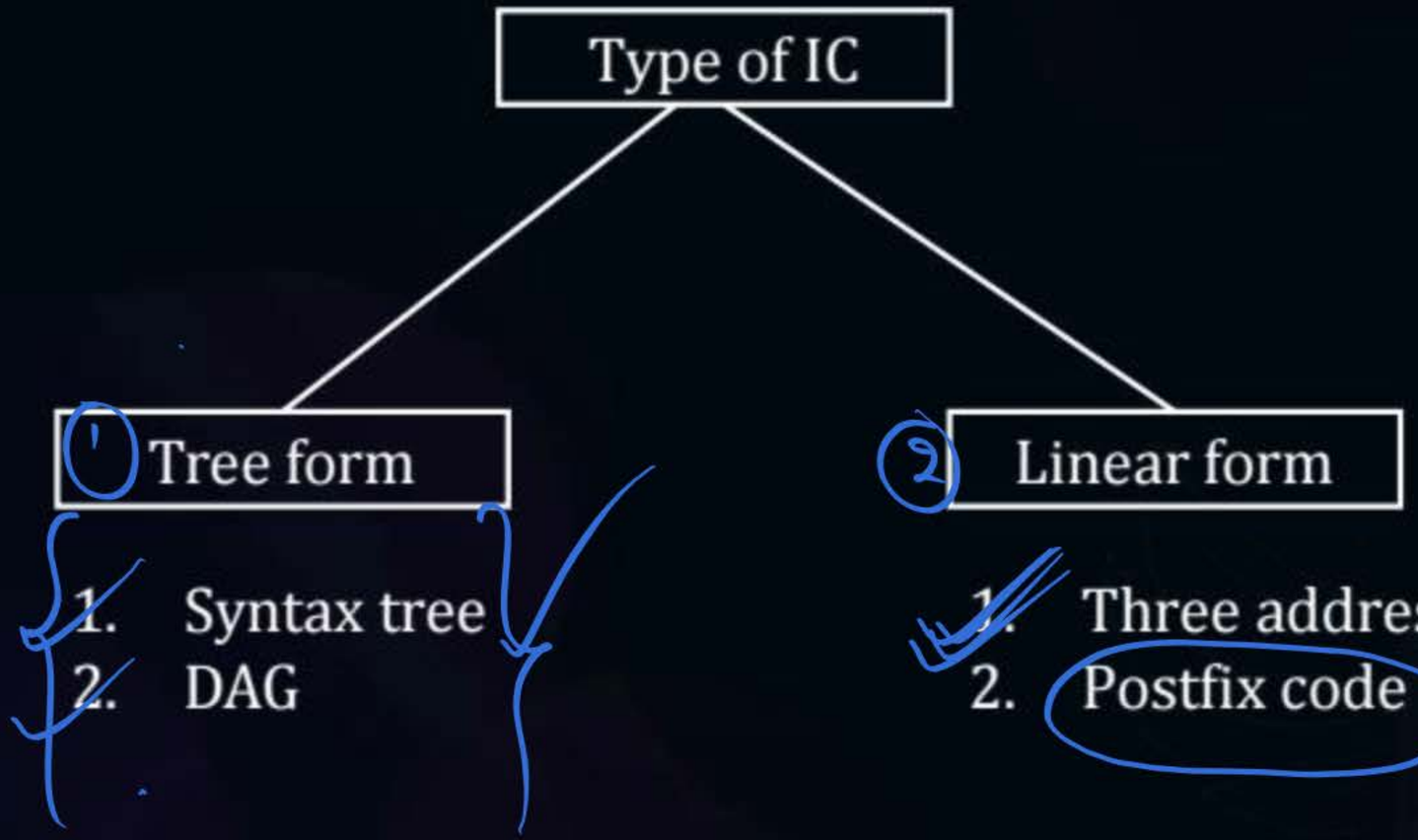
Topic : Introduction of Compiler Design



Topic : Intermediate Code Generation

a + b * c

a b c * +



Topic : Intermediate Code Generation

Three Address Code:

TAC is an intermediate code in which high level language statement can be represented by using atmost 3 address

↓ ↓ ↓

ADD R_{01} R_1

$t_1 = a + b$

ADD R_0, R_1

Topic : Intermediate Code Generation

$t_1 := y + z$ $t_1 := -y$

1. x = yopz
2. X = opY
3. x = y

} Expression

4. x[i] = y
   x = y[i]

} arrays

5. Goto L } unconditional jump
6. If a > b goto L_1
   goto L_2

} Conditional jump

} Flow control

if, else, for

Topic : Intermediate Code Generation

7. Procedure/function

Param P_1

Param P_2

.

.

.

Param P_n

call name, number of arguments

Topic : Intermediate Code Generation

1. $x = a + b * c$

TAC

$t_1 = b * c$

$t_2 = a + t_1$

$x = t_2$

TAC

$t_1 = b * c$

$t_2 = a + t_1$

$x = t_2$

Topic : Intermediate Code Generation

2. $x = (a * b) + (c * d)/(a * b)$

How many temporary variable generated

$t_1 = a * b$

$t_2 = c * d$

$t_3 = a * b$

$t_4 = t_2 / t_3$

$t_5 = t_1 + t_4$

$x = t_5$

Topic : Intermediate Code Generation

3. $x = a * b + c * d - e * f$

How many temporary variable generated to construct TAC.

$t_1 = a * b$ ✓

$t_2 = c * d$ ✓

$t_3 = e * f$ ✓

$t_4 = t_2 + t_1$

$t_5 = t_4 - t_3$ ✓

$x = t_5$ ✓

} Temp

Static Single Assignment

Topic : Intermediate Code Generation

Static Single Assignment: [also TAC]

In static single assignment every expression is assigned to new variable.

Reusing of variable is not allowed in static single assignment.

(ii) $X = \underbrace{\underbrace{(a+b)}_{t_1} * \underbrace{(a+b)}_{t_1}}_{t_2} * (a+b)$ How many minimum temporary variables required in 3-address code

$$\begin{cases} t_1 = a+b \\ t_2 = t_1 * t_1 \\ t_1 = t_2 * t_1 \\ X = t_1 \end{cases}$$

2 temporary

(Q) $q + r/3 + s - t * 5 + u * v/w$ } How many minimum temporary variables required in Static Single allignment

(Q) $9 + (r/3) + s - (t * 5) + (u * v)/w$

t_1 t_2 t_3 t_4 t_5 t_6

How many minimum temporary variables required in Static Single allignment

$t_1 = r/3$ ✓

$t_2 = t * 5$ ✓

$t_3 = u * v$ ✓

$t_4 = t_3/w$ ✓

$t_5 = 9 + t_1$ ✓

$t_6 = t_5 + s$ ✓

$t_7 = t_6 - t_2$ ✓

$t_8 = t_7 + t_4$ ✓

8 temporary

Topic : Intermediate Code Generation

if else

Construct TAC for flow Control Statement

```
if a > b
{
L1: x = y + z;
    }
    else
    {
L2  A = B + C
}
```

TAC

```
if a > b goto L1
goto L2

L1: t1 = y + z
    x = t1
    goto LAST

L2: t1 = B + C
    A = t1
    goto LAST

LAST:
```

Topic : Intermediate Code Generation

Flow Control

if a > b goto L_1

goto L_2

L_1 : $t_1 = y + z$

$x = t_1$

L_2: $t_1 = B + C$

$A = t_1$

goto LAST

Topic : Intermediate Code Generation

#Q. Construct TAC for following for Loop.

```
for (i = 1; i < 100; i + + )
{
    x = y + z * a;
}
```

i+1

L_2:

i = 1;

L_0: if i<100 goto L_1
goto L_2

L_1: $t_1 = z * a$
$t_2 = y + t_1$
$x = t_2$

$t_3 = i + 1$
$i = t_3$
goto L_0

L_2:

Topic : Intermediate Code Generation

```
        i = 1
        if i < 100 goto L1
        goto Last
L1:     t1 = A * A
        t2 = y + t1
        x = t2
        t3 = i + 1
        i = t3
```

Topic : Intermediate Code Generation

1D Array

$$A[i] = base + (i - low) * W$$

2D Array

$$A[i, j] = [(i * n_2 + j) * W] + base - [(low_1 * n_2) + low_2] * W$$

Topic : Intermediate Code Generation

#Q. Construct TAC for the following array

$x = A[i, j]$

size of array A[10][20], $low_1 = low_2 = 1$

Size of each word is 4 bytes.

$$A[i, j] = (i * n_2 + j) * W + base - [(low_1 * n_2 + low_2)] * w$$

$$= (i * 20 + j) * 4 + base - [1 \times 20 + 1] * 4$$

$$A[i, j] = (i \times 20 + j) * 4 + base - 84$$

$t_1 = i \times 20$

$t_2 = t_1 + j$

$t_3 = t_2 * 4$

$t_4 = base - 84$

$t_5 = t_3 + t_4$

$x = A[t_5]$

Topic : Intermediate Code Generation

Quadruple

$x = a + b * c$

$t_1 = b * c$

$t_2 = a + t_1$

$x = t_2$

$t_1 = b * c$

$t_2 = a + t_1$

$x = t_2$

Result	Op	Op_1	op_2
t_1	*	b	c
t_2	+	a	t_1
x	=	t_2	

Topic : Intermediate Code Generation

Triples:

	Op	Opr_1	opr_2
(100)	*	b	c
(104)	+	a	(100)
	=	x	(104)

$t_1 = b * c$

$t_2 = a + t_1$

$x = t_2$

Topic : Intermediate Code Generation

Indirect triples

100	(500)
(104)	(504)

	Op	Opr_1	Opr_2
(100)	*	b	c
(104)	+	a	(100)
	=	x	(104)

Topic : Triple

$X = a + b * c$

$t_1 = b * c$

$r_2 = a + t_1$

$X = t_2$

	Operator	OPr_1	OPr_2
(100)	*x*	*b*	*c*
(104)	+	*a*	*(100)*
	=	*x*	*(104)*

Topic : Quadruple

$X = a + b * c$

1

$t_1 = b * c$ — 1

$r_2 = a + t_1$ — 2

$X = t_2$ — 3

X

result	Operator	OPr_1	OPr_2
t_1	*	b	c
t_2	+	a	t_1
X	=	t_2	-

Topic : Indirect Triple

fixed

(500) →	(100)
(600) →	(104)

	Operator	OPr_1	OPr_2
(100)	x	b	c
(104)	+	a	(100)
	=	x	(104)

Topic : Optimization

1. Constant folding
2. Constant Propagation
3. Variable Propagation
4. Common subexpression elimination
5. Strength Reduction
6. Dead Code elimination
7. Loop Invariant Computation
8. Loop Unrolling
9. Induction Variable

Topic : Optimization

Example:-

```
X = 2 * 4
```

Constant Folding

```
X = 8;
```

Topic : Optimization

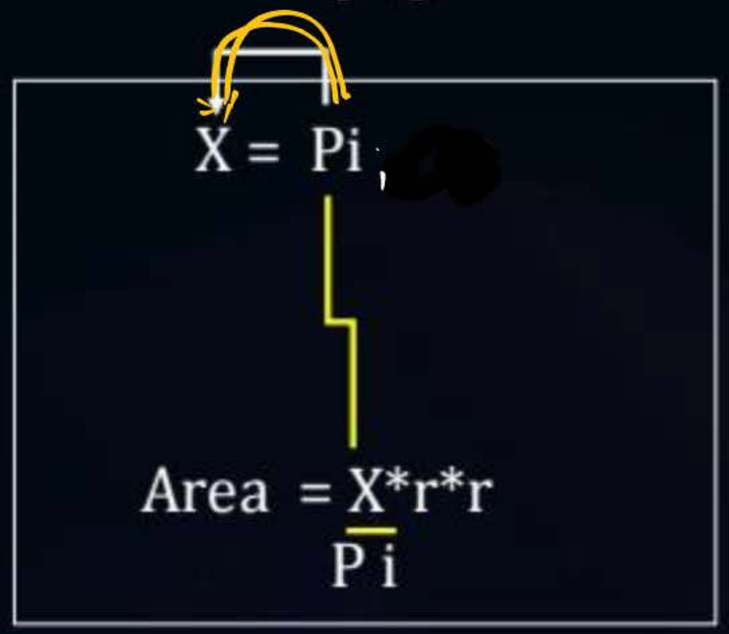
Constant Propagation

```
Pi = 3.14
Area = pi*r*r*
```

```
Area = 3.14 * r * 
```

Topic : Optimization

Variable Propagation



Area = pi *r *r*

Topic : Optimization

{DAG}

Common Sub expression elimination

$X = (a + b) * (c * d) / (a + b)$

$t_1 = a + b$

$t_2 = c * d$

$t_3 = a + b$

$t_4 = t_2 / t_3$

$t_5 = t_1 * t_4$

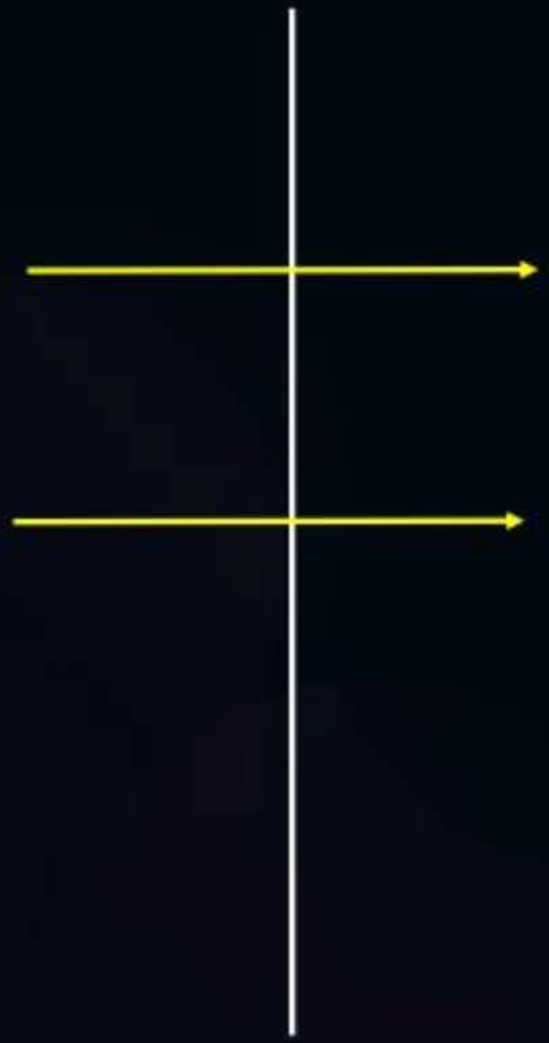
$t_1 = a + b$ ✓

$t_2 = c * d$ ✓

$t_3 = t_2 / t_1$

$t_4 = t_1 * t_3$

Topic : Strength Reduction



X = 2 * a;

X = a + a; ✓

Y = 3 * z;

Y = z + z + z; ✓

Topic : Dead Code Elimination

```
i = 0
If (i = = 1)
{
X = y + z;
}
```

Dead Code

Topic : Optimization

Loop Invariant Computation

```
While ( i < = max –1)
{
Sum = sum + a [i]
}
```

```
n = max –1
While ( i < = n)
{
Sum = sum + a [i]
}
```

Topic : Loop Unrolling

```
Int    i = 1
While (i < 100)
{
      a [i] = b [i];
      i ++
}
```

100

```
Int    i = 1
While (i < 100)
{
      a [i] = b [i];
      i ++
a [i] = b [i],
i++
}
```

50

Induction Variable

```
for(i=0; i<n; i++)
  {
    a= k+y*b;
     b= b+ i
  }
```

Induction Variable

```
for(i=0; i<n; i++)
{
    a= k+y*b;
    b= b+i
}
```

Induction Variables

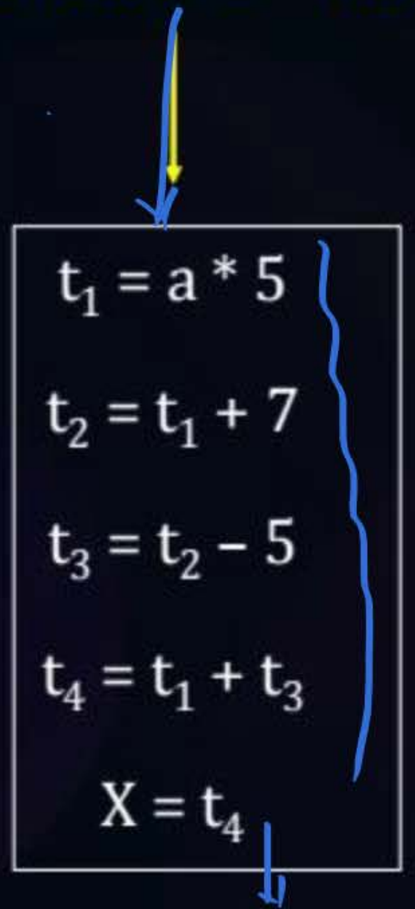
i, a, b

Invariant Variables

n, y, k

Topic : Basic Block

Basic Block: Basic Block is a sequence of three address code Instruction in which flow of control entres at the beginning and leaves at the end.



Topic : Basic Block

Leader statements

Algorithm for Basic Block construction

1. First statement is a leader
2. Target of conditional (or) Un conditional goto is a leader [goto target]
3. Any statement that immediately follow goto is a leader. (next)

Leaders = {1, 3, 11}

B_1

1. Print
2. i = 1

B_2

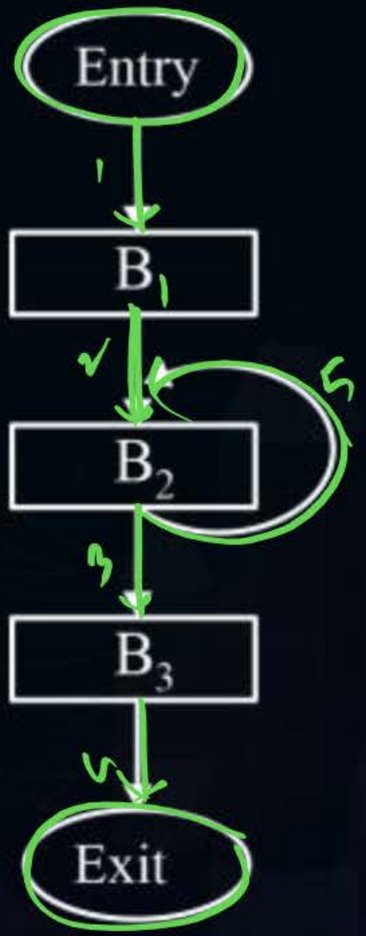
3. $t_1 = i * 4$
4. $t_1 = i * 4$
5. $t_3 = 4 * i$
6. $t_4 = b[t_3]$
7. $t_5 = t_3 * t_1$
8. $t_6 = prod * t_5$
9. Prod $= t_6$
10. If i < 20 goto (3)

B_3

11. X = y + Z
12. Stop

Construct Basic block

Control flow Graph



nodes: 5

edges: 5 edges

Leader = {1, 3, 4, 8}

B_1

1. f = 1
2. i = 2

B_2

3. If i < n go to (8)

B_3

4. f = f * i
5. t = t + 1
6. i = t
7. goto (3)

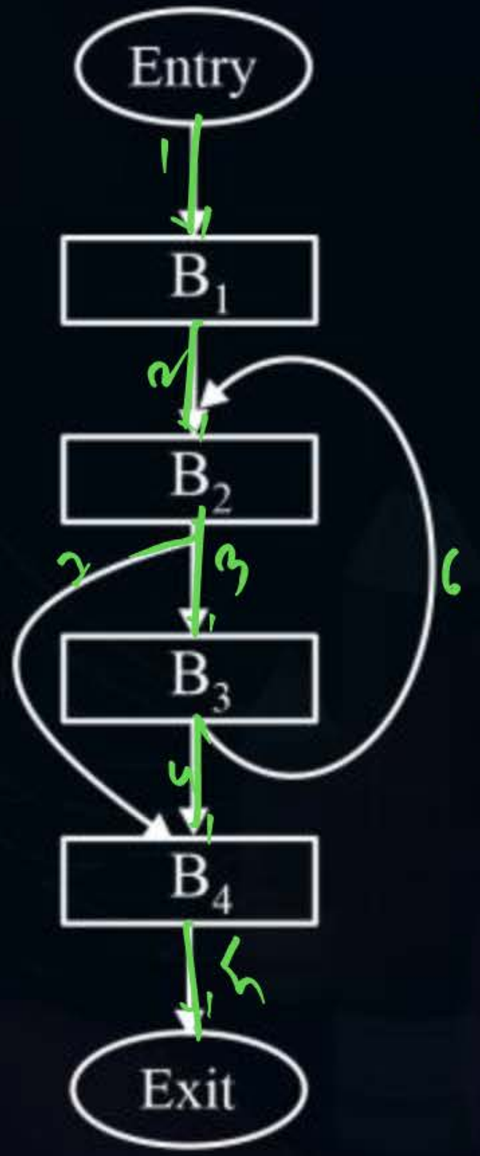
B_4

8. i = 5 + j
9. i = I + 1

6 nodes

7 edges

Control flow Graph



nodes = 6

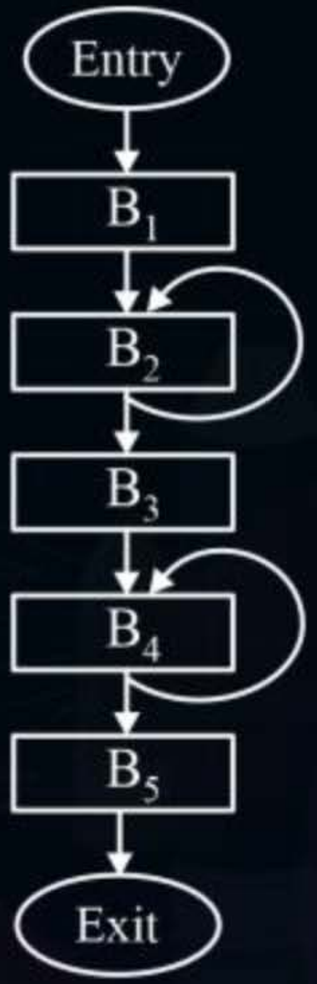
edges = 7

Leaders
1, 5, 8, 9, 12

B_1	1.	i = m + 3
	2.	K = n
	3.	$t_1 = 4 * n$
	4.	i = i * 1
B_2	5.	$t_2 = 4 * i$
	6.	$t_3 = a [t_3]$
	7.	If $t_3 < n$ goto (5)
B_3	8.	K = k –1
B_4	9	$t_4 = 4 * k$
	10.	$t_5 = 0 [t_4]$
	11.	If $t_5 < n$ goto (9)
B_5	12.	i = i + 1
	13.	$X = y + z_1$

Construct Basic block

Control flow Graph



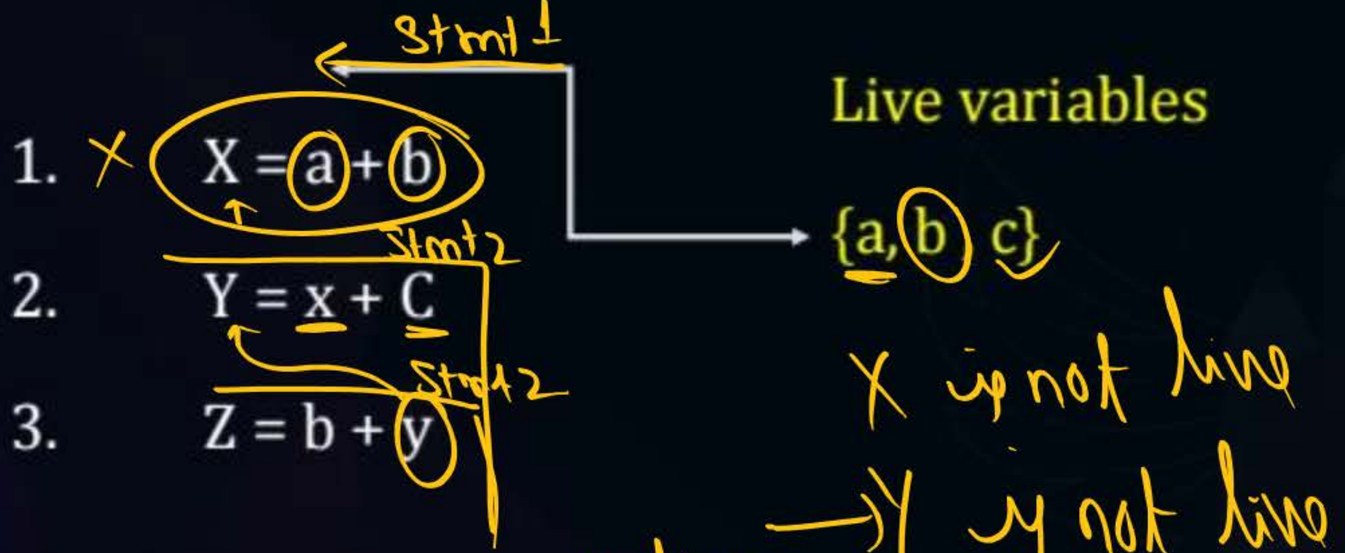
7 nodes

8 edges

Topic : Liveness Analysis

Live Variable: A variable is said to be live at some point p if the value of x is used before it if redefined.

① used



X is live at statement S_i iff

1. There is a statement Sj that reads x (used)
2. There is a path between Si and Sj
3. Now write to x before Sj

(redefined)

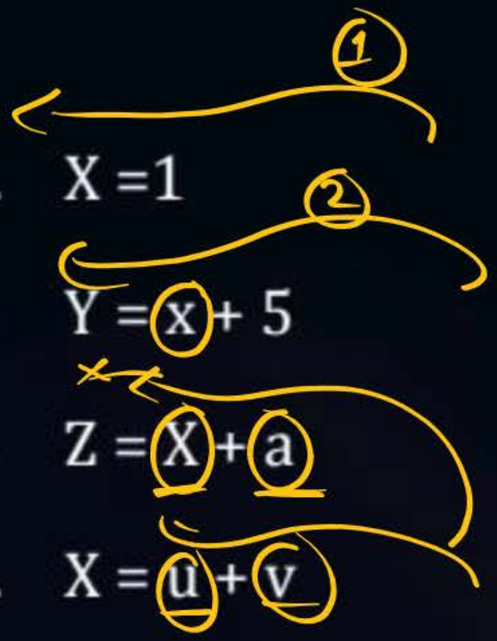
Topic : ???

1. $X = a + b$
2. $Y = X + C$
3. $Z = Z * a$
4. $X = Y + Z$

X	Y	Z	a	b	c
Not live	Not live	live	Live	Live	Live
Live	Not live	Live	Live	Not live	Live
Not live	Live	Live	Live	Not live	Not live
Not live	Live	Live	Not live	Not live	Not live

Topic : Liveness Analysis

Stmt 1 = {a, u, v}



1. Live variable at statement 2 {x, a, u, v}
2. Live variable at statement 3 {x, a , u, v}
3. Live variables at statement 4 {u,v}

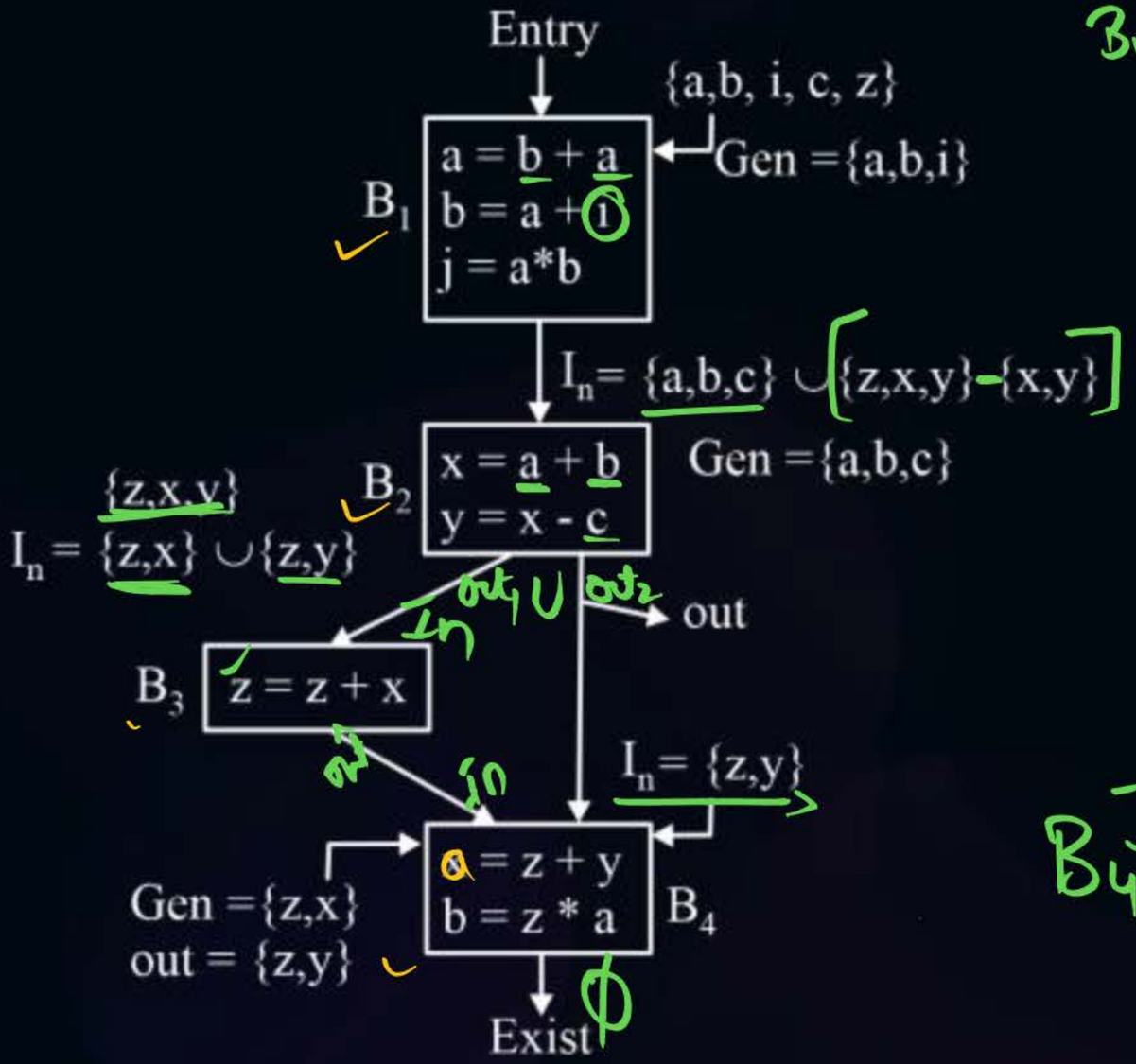
Data flow Analysis

Data flow Equations

Topic : Liveness Analysis

1. GEN / Use set } Live variables in Basic Block.
2. KILL / Def set
3. In;
4. Out;

Backward Analysis



I_n : Gen ∪ [out - kill]

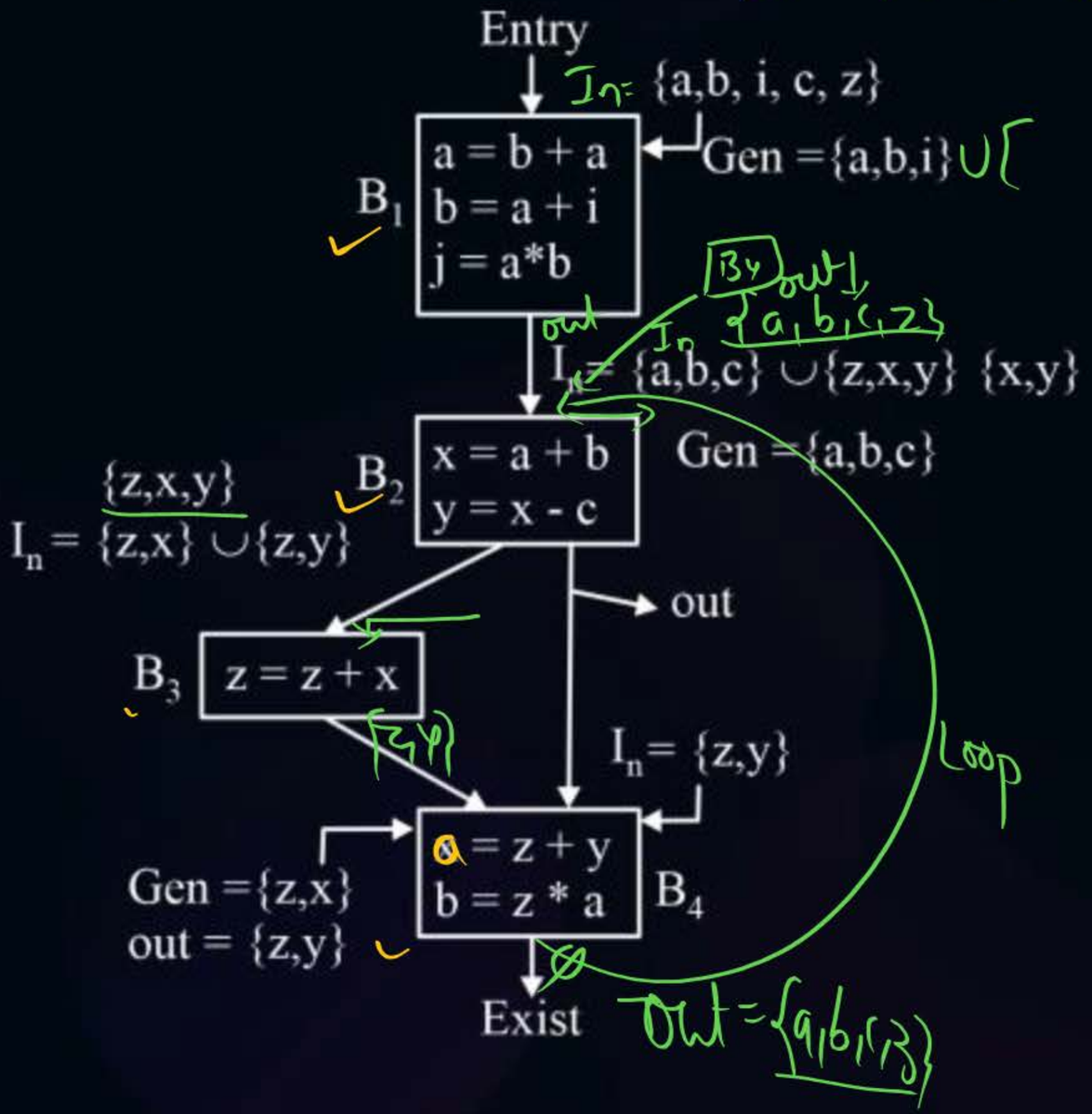
$B_4 I_n$: {z,y} ∪ {o - y a, b}

	Gen set	Kill set	Out	In
B_1	{a,b,i}	{a, b, j}		
B_2	{a,b,c}	{x,y}	{z,x,y}	
B_3	{z,x}	{z}	{z,y}	{z,x,y}
B_4	{z,y}	{a,b}	ϕ	{z,y}

$B_4 I_n$ = {z,x} ∪ [(z,y) - z)]

{a,b,i} ∪ {a, b, c,z} – {a, b, j}

Backward Analysis



	Gen	Kil	Out	In
B_1	{a,b,i}	{a, b, j}	{a,b,c,z}	{a,b,i,c,z}
B_2	{a,b,c}	{x,y}	{z,x,y}	{a,b,c,z}
B_3	{z,x}	{z}	{z,y}	z,x,y}
B_4	{z,y}	{a,b}	ϕ	{z,y}

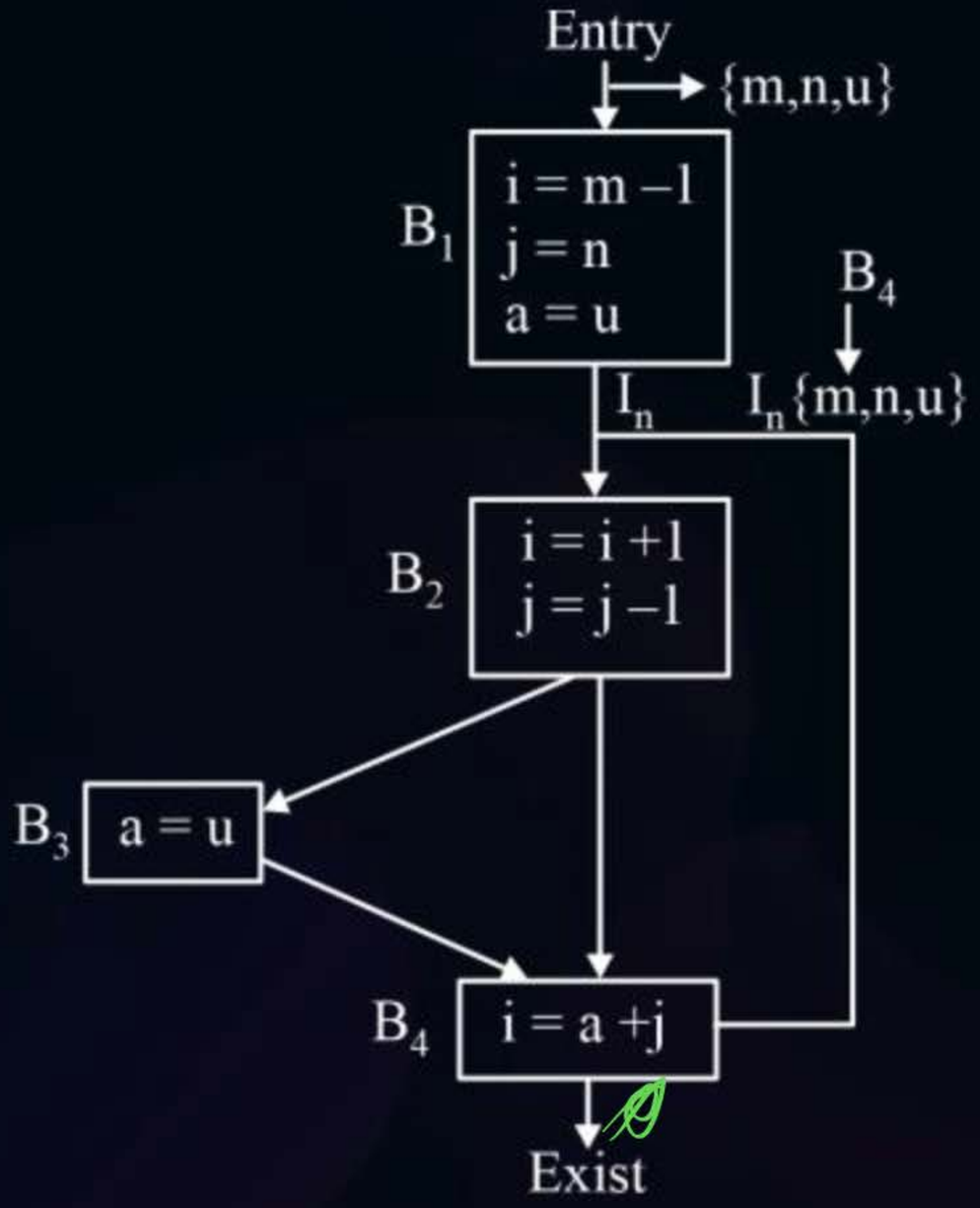
{a,b,i} ∪ {a, b, c,z} – {a, b, j}

$$\text{In} = \text{Gen} \cup (\text{Out} - \text{Kill})$$

$$In_{B_4} = \{a, j\} \cup (\phi - i)$$

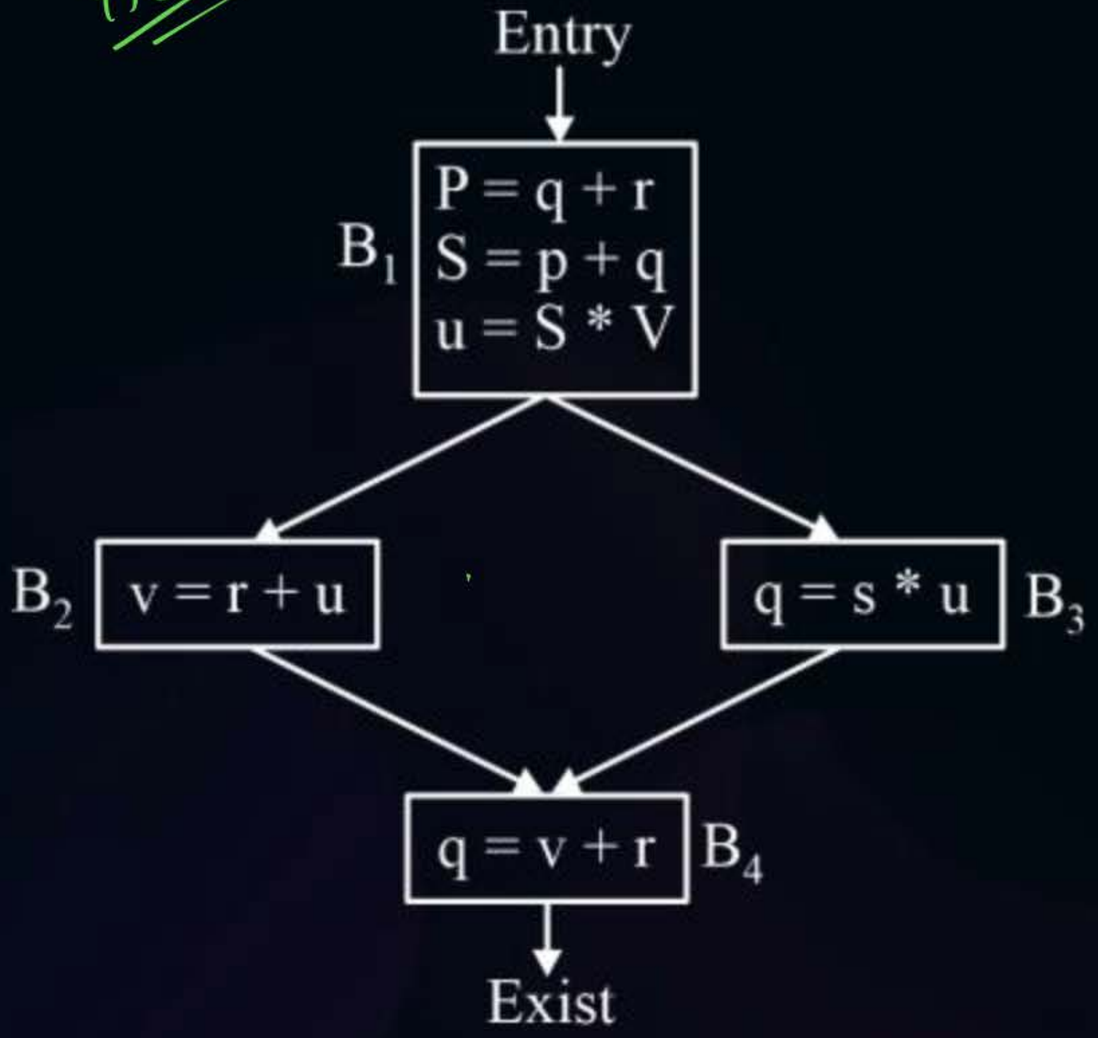
$$In_{B_4} = \{a, j\}$$

$I_n = Gen \cup [(out - kill)]$



	Gen	Kil	Out	In
B_1	{m,n,u}	{i, j, a}	{u,a,i,j}	{m,n,u}
B_1	{i,j}	{i,j}	{u,a, j}	{u,a,i,j}
B_3	{u}	{a}	{a,j}	{u, i, j}
B_4	{a,j}	{i}	ϕ	{a,j}

Home Work



	Gen	Kil	Out	In
B_1				
B_1				
B_3				
B_4				

Topic : Run Time Environment

Run Time memory

Topic : Runtime Environment

Storage Allocation Strategies

1. Static allocation
2. Stack allocation
3. Heap allocation

Topic : Runtime Environment

Static Storage Allocation

1. Size of data object known at compile time. Hence bound to storage at compile time only.
2. Compiler can determine amount of storage required at compile only
3. No Runtime storage support
4. Dynamic data Structure can't be created.

Topic : Runtime Environment

main()
{
E()
F();
}

Stack Allocation

1. Storage is Organized as a stack.
2. For every function call activation record is pushed into stack. Once function returns its value activation record is popped out from stack.
3. Runtime memory management is automatic

Topic : Runtime Environment

Heap Allocation

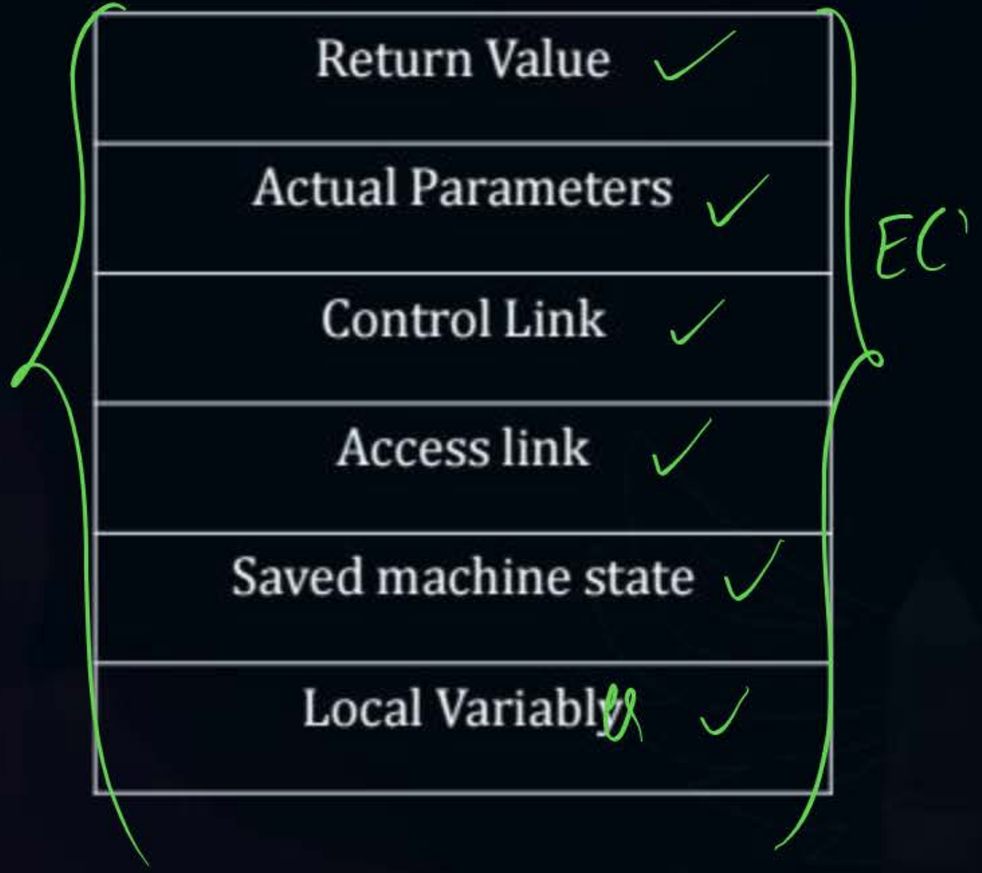
1. Memory is allocated as continuous block of memory
2. Heap memory management is overhead

malloc()

free()

Topic : Activation Record

Activation Record



(1) Which languages necessarily need heap allocation in the runtime environment?

(a) Those support Recursion ✓

(b) Those use global variables ✗

(c) Those use dynamic data structure

(d) none

2 mins Summary

(24-6)

- Topic One
- Topic Two
- Topic Three
- Topic Four
- Topic Five

Parsing

SDT

{Lexical, ICG}

{Optimization}

THANK - YOU